Agentic FedGrid

Autonomous Privacy-Preserving Intrusion Detection via Federated Learning and Agentic AI

The complete explainer: what we built, every model and algorithm, the backend, the tech stack, how the live demo works, and what the results really say.

Team	Mohith Kumar M (1JS23IS090) · Anusha B (1JS23IS016) · Lokesh Kumar Yadav (1JS22IS065) · Rohan C Ravi (1JS24IS410)
Guide	Dr. Naidila Sadashiv
Department	Information Science and Engineering, JSS Academy of Technical Education, Bengaluru · BIS786 · 2025-26
Live demo	https://fedgrid.mohithkumar.dev

1. The one-minute version

We run **four intrusion-detection nodes on AWS**, each sitting on the public internet. Each node watches the packets hitting it, uses a small **Transformer neural network** to decide how likely a connection is to be an attack, and then an **agent** decides what to do: ignore it, raise an alert, or write a real firewall rule that blocks the attacker. Nobody tells it what to do; it decides on its own.

The four nodes also **learn together without sharing their data** (this is *federated learning*). Every two minutes each node trains on its own private traffic and uploads only a tiny 10 KB set of weight changes (a *LoRA adapter*) to the cloud. The cloud averages them (*FedAvg*) and sends the improved model back. Each node tests the new model on its own data and refuses it if it makes things worse.

And they **warn each other**: when one node blocks an attacker, it posts the attacker's IP to the cloud and the other three block it too, typically within about 9 seconds, before that attacker ever touches them.

Privacy rule of the whole project: raw traffic never leaves a node. Only model weights and "this IP is bad" verdicts cross the network.

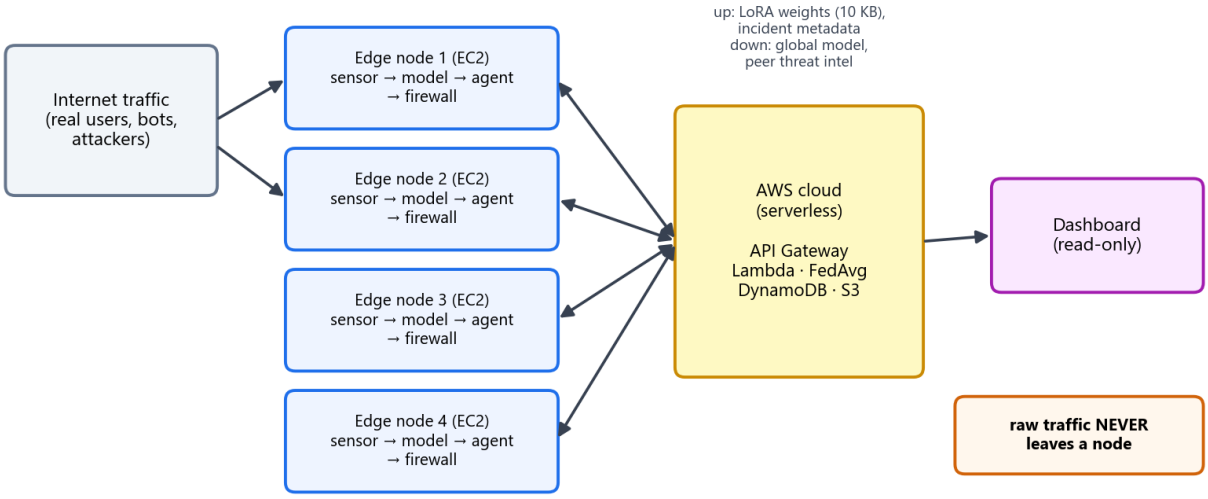


Figure 1. The big picture: 4 edge nodes, a serverless AWS control plane, and a read-only dashboard.

2. Why this project exists (the problem)

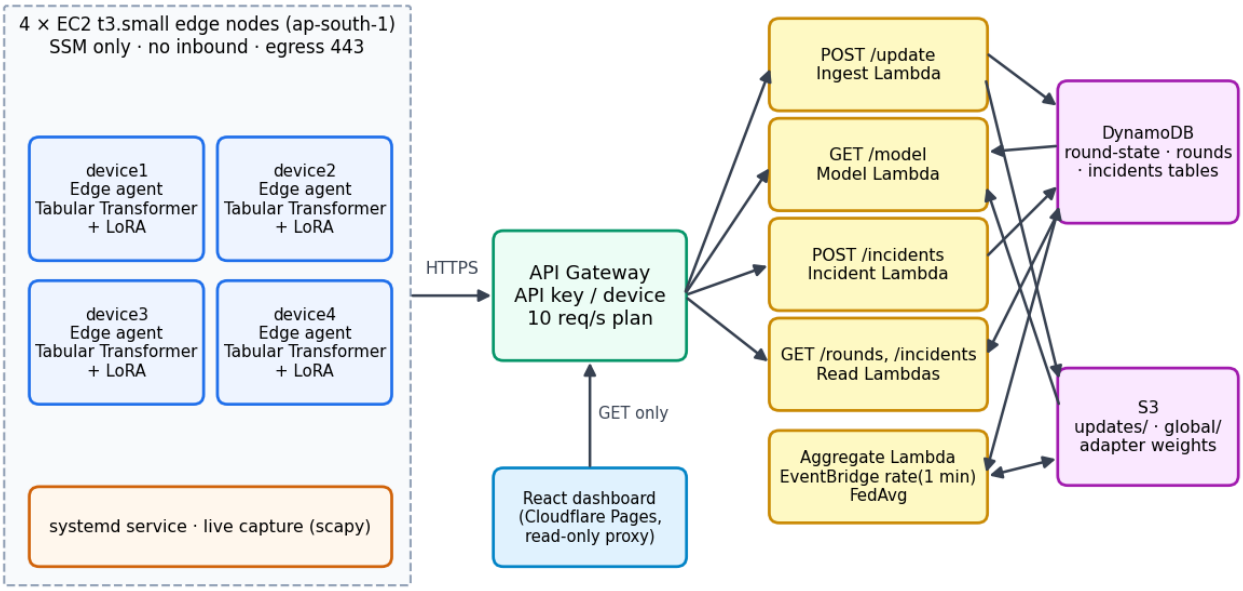
- **Central IDS = privacy problem.** Classic intrusion detection ships all traffic to one server. That is a privacy, bandwidth and latency risk.
- **Each site only sees its own attacks.** An attack that hit a college network yesterday is unknown to a hospital network today.
- **Classifiers alone are dangerous.** Most research stops at "accuracy 99%". In reality, a model that blocks on every false positive cuts off legitimate users. What the system *does* with a verdict matters as much as the verdict.
- **Federated learning has its own risks:** full-model updates are big, data is uneven across sites (non-IID), and one malicious site can poison the shared model.

Our answer combines three ideas: **federated learning** (learn together, keep data local), **parameter-efficient fine-tuning** (LoRA, so updates are tiny), and an **agent** that reasons about the cost of each action and can refuse bad models.

3. System architecture

The system has six layers. Each one maps to real code in the repository.

Layer	What it does	Where in the code
Edge device	Capture traffic, run the model, decide, firewall, train locally	edge_agent/*.py, run_edge_node.py
Communication	HTTPS to API Gateway; only adapter weights + metadata; one API key per device	edge_agent/comms.py
Cloud orchestration	Round coordination, FedAvg, storing models and incidents	cloud/*_lambda, cloud/shared, cloud/template.yaml
Model	Frozen Tabular Transformer backbone + per-device LoRA adapter	model/tabular_transformer.py
Defense	Graded iptables blocks + shared threat-intel channel	edge_agent/actuator.py, agent.py
Dashboard	React SOC console, read-only, on Cloudflare Pages	dashboard/



Only LoRA adapter weights (~10.3 KB) and incident metadata leave the node; raw flows never do. Arrows show data flow.

Figure 2. The AWS deployment as it actually runs (arrows show data flow; matches cloud/template.yaml).

4. The data

We train on two public benchmark datasets from the same research group (UNSW Canberra), which use the same flow-feature pipeline, so their columns line up:

- **UNSW-NB15**: enterprise-style network traffic with 9 attack families (DoS, Exploits, Fuzzers, Reconnaissance, Backdoor, Shellcode, Worms, Analysis, Generic).
- **ToN_IoT**: IoT network traffic with attacks such as DDoS, ransomware, XSS, injection, password attacks, scanning and MITM.

The 8 canonical features. We keep only the fields both datasets share (no made-up imputation): 5 numeric (*duration, src_bytes, dst_bytes, src_pkts, dst_pkts*) and 3 categorical (*proto, service, state*). The categorical vocabulary is built once from both datasets and shipped identically to every node; otherwise the adapters could not be averaged.

Non-IID split. Devices 1-2 get UNSW-NB15 shards and devices 3-4 get ToN_IoT shards ("enterprise sites" vs "IoT sites"). Within a dataset, rows are split with a **Dirichlet(α = 0.5) label-skew partition** (Hsu et al. 2019): each attack class is spread across clients in random, uneven proportions, so every node sees a different mix. This is deliberately harder than a random split.

The "zero-day" hold-out. One attack class, **Fuzzers**, is removed from central pretraining entirely and given only to device 1. This lets us test the key question: *can federation teach the other nodes an attack only one site has seen?* (Section 9 gives the honest answer.)

5. The models

5.1 Tabular Transformer (the detector)

Transformers are usually used for text. Here each feature group becomes a **token**: the 5 numeric features are projected into one 32-dimensional token, and each of the 3 categorical features gets its own learned 32-d embedding. These 4 tokens go through a **2-layer Transformer encoder** (4 attention heads, feed-forward 32→128→32), and a linear head outputs two scores, turned into **P(attack)** with a softmax. Self-attention lets the model learn interactions such as "this protocol with this byte pattern".

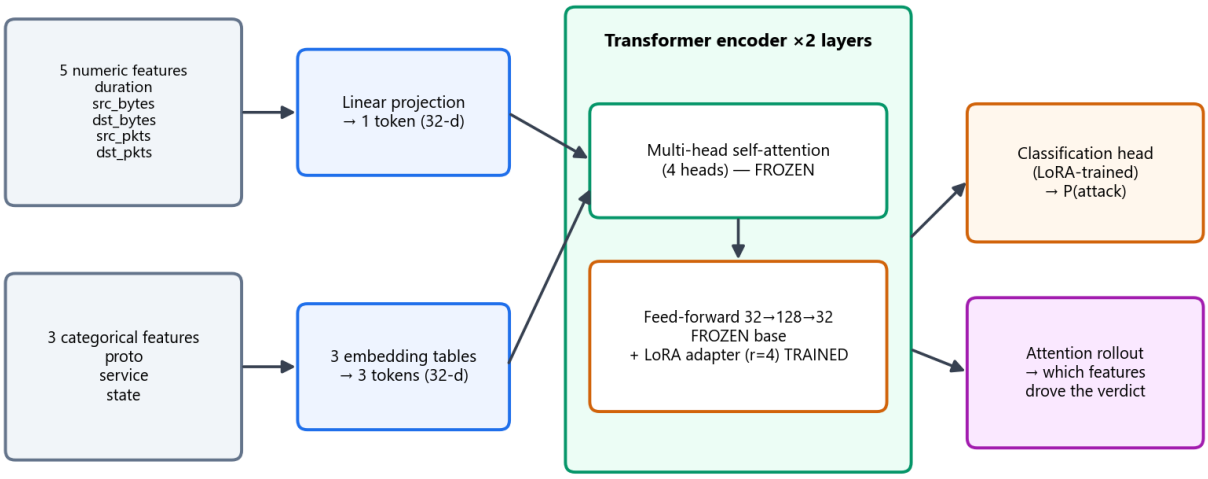


Figure 3. The detector. Only the orange parts (LoRA adapter and head) are trained on the edge.

Training. The backbone is pretrained once, centrally, with Adam (lr 1e-3, OneCycle schedule, batch 256, 10 epochs) on everything except Fuzzers. It is then **frozen** and shipped to all 4 nodes.

5.2 LoRA adapters (what is federated)

Instead of retraining the whole network on each node, we add a small **Low-Rank Adaptation** (Hu et al. 2021) next to each feed-forward layer. A frozen weight matrix W gets a trainable low-rank correction:

```
W' = W + (α / r) · B · A A: r × d_in, B: d_out × r, r = 4, α = 8
```

B starts at zero, so the adapter begins as "no change". Only A, B and the classification head are trained and uploaded. Effect: **10.3 KB per upload instead of 103.7 KB** for the full model (10× smaller). We use HuggingFace **peft** to apply it, only *after* pretraining.

5.3 Local training on each node

- **Budget:** 5 epochs × 20 steps, batch 64, Adam on adapter parameters only. Small on purpose, so it is a nudge, not a full retrain.
- **Hard-example mining:** samples the model currently gets wrong are drawn more often (loss-weighted, capped at 3× with a floor of 0.3), so rare attacks aren't drowned by benign traffic.

- **Adversarial training (FGSM):** each step also trains on slightly perturbed inputs, so an attacker can't dodge detection by nudging features:

```
x_adv = x + ε · sign( ∇_x Loss(x, y)), ε = 0.05
Loss_total = Loss(x, y) + Loss(x_adv, y)
```

- **Self-validation:** 15% of local data is held out and never trained on; the node reports its accuracy there ("federated validation accuracy", not a test score).

5.4 Attack-type classifier (bonus model)

A second, two-stage model names the attack: stage 1 is the binary detector above (normal vs attack), stage 2 is a type head trained on attacks only. On UNSW-NB15 it reaches **74.8% accuracy but only 55.0% balanced accuracy**, because rare classes are hard. Honest status: useful as a label on the dashboard, not a headline result.

5.5 Explainability: attention rollout

For every detection we compute **attention rollout** (Abnar & Zuidema 2020): multiply the attention matrices of both layers (adding the identity for residual connections) to estimate how much each input token influenced the final decision. The agent logs these weights with each incident, so an analyst can see *why* it blocked (for example "src_pkts and state drove this").

```
R = (0.5·A_2 + 0.5·I) · (0.5·A_1 + 0.5·I) (rows re-normalised)
```

5.6 Reference models (for honesty)

We also trained a **gradient-boosted tree** model on the same 8 features. It scores **90.1%** on the official UNSW-NB15 test set vs **87.7%** for our Transformer. We report this openly: the contribution of the project is the federated, agentic system, not beating trees on accuracy.

6. The agent: why it is "agentic" and not just if-else

The classifier, the aggregator and the firewall are **tools**. The **EdgeAgent** (edge_agent/agent.py) is the controller that runs a **perceive → infer → decide → act → reflect** loop and chooses which tool to use. Two nodes running the identical model can make different decisions at the same moment, because their local context differs (recent traffic, their own blocklist, their own feedback).

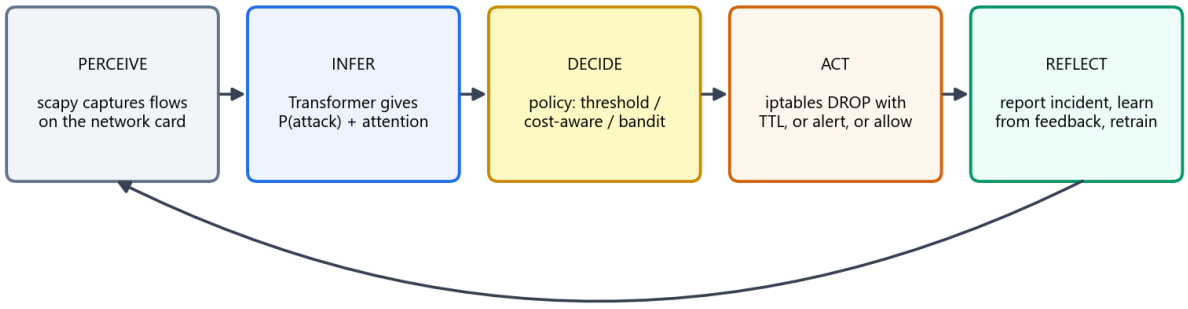


Figure 4. The agent loop on every node.

6.1 Decision policies (three, compared in the paper)

(a) Threshold rule (baseline): BLOCK if P(attack) ≥ threshold (0.65 on the live demo nodes), else ALERT/ALLOW.

(b) Cost-aware rule (Bayes decision rule): pick the action with the lowest *expected cost*. With p = P(attack):

```
E[cost | a] = (1 − p) · cost[a][benign] + p · cost[a][attack]
choose argmin over a   {NOOP, ALERT, BLOCK}
```

Action	Cost if flow is benign	Cost if flow is an attack
NOOP (allow)	0	10 (missed attack)
ALERT	1 (analyst glance)	3
BLOCK	20 (outage for a real user)	0

This makes the trade-off explicit: blocking a legitimate user is treated as 20× worse than an analyst glancing at an alert. If an analyst marks a block as a false positive, that action's benign cost goes up (the agent becomes more careful); a confirmed block lowers it slightly.

(c) LinUCB contextual bandit (Li et al. 2010), the learning policy. For each flow it builds a context vector x (P(attack), the source's recent behaviour from FlowHistory such as connection-state mix and host/service fan-out, and whether a peer already blocked it) and picks one of NOOP / ALERT / BLOCK / INVESTIGATE:

$$\text{score}(a) = \theta_a^{\top} x + \alpha \cdot \sqrt{x^{\top} A_a^{-1} x}, \quad \theta_a = A_a^{-1} b_a$$
$$A_a = \lambda I + \Sigma\, x\, x^{\top}, \quad b_a = \Sigma\, r \cdot x, \quad \text{reward } r = -\text{cost} / 20$$

The first term is "what has worked before", the second is an exploration bonus for actions it is unsure about. Because A and b are plain sums, **federating the bandit is exact**: adding the statistics from several nodes gives the same bandit one node would get from all the feedback.

6.2 Graded, reversible blocking

Blocks are not permanent on the first offence. Level k blocks for **60 s × 2^{k-1}** (60 s, 120 s, 240 s…) and become permanent after 4 escalations. Expired blocks are lifted automatically, and an analyst rollback removes a block and creates a short exception for that IP.

6.3 Model-adoption gate

When a new global model arrives, the node evaluates it on its own held-out data and adopts it only if accuracy does not drop by more than 1 point and false-positive rate does not rise by more than 2 points. This is what lets a node **refuse a poisoned or harmful update**. On the live dashboard this shows up as REJECT_GLOBAL.

6.4 Peer threat intelligence

A second, much faster collaboration channel next to FedAvg: every BLOCK is posted as an incident (IP, verdict, confidence, attention), and every node polls recent incidents every 5-15 s and pre-emptively blocks IPs its peers blocked. Measured propagation: **median ≈ 9 s, worst ≈ 12 s**.

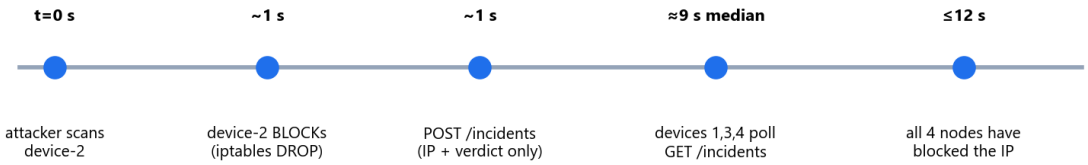


Figure 5. One node is attacked, all four are protected.

6.5 LLM investigator (built, not live)

For uncertain flows (the cost rule's ALERT band), edge_agent/investigator.py can hand the case to Claude with tools (source profile, recent flows, threat intel) and get a block/alert/allow verdict. It is implemented and evaluated in simulation, but **switched off in the live deployment** (INVESTIGATE=0) because Bedrock access was never enabled. Don't present it as running live.

7. The backend (AWS, serverless)

The cloud never runs the neural network. It only coordinates rounds and averages numbers, so no PyTorch in Lambda. Weights travel as **base64-encoded float32 arrays in JSON** (no pickle, so nothing executable crosses the wire). Everything is defined as code in an **AWS SAM** template (stack *fedgrid*, region ap-south-1 Mumbai).

Component	Role
API Gateway (REST)	Single HTTPS entry point. One API key per device; usage plan 10 req/s, burst 20.
Ingest Lambda — POST /update	Saves the adapter to S3, joins the device to the current round in DynamoDB; runs FedAvg inline when all 4 have reported.
Model Lambda — GET /model	Returns the latest global adapter from S3.
Incident Lambda — POST /incidents	Stores a block/alert incident (IP, verdict, confidence, attention).
Rounds / Incidents read Lambdas	GET /rounds and GET /incidents for the dashboard and for peer intel.
Aggregate Lambda (EventBridge, every 1 min)	Straggler handling: if a round has been open past 180 s with ≥ 2 of 4 updates, aggregate with whoever reported.
DynamoDB (3 tables)	round_state (single GLOBAL item: round, submitted set, status), rounds (per-round metrics), incidents.
S3 bucket	updates/roundN/deviceX and global/roundN adapter files.

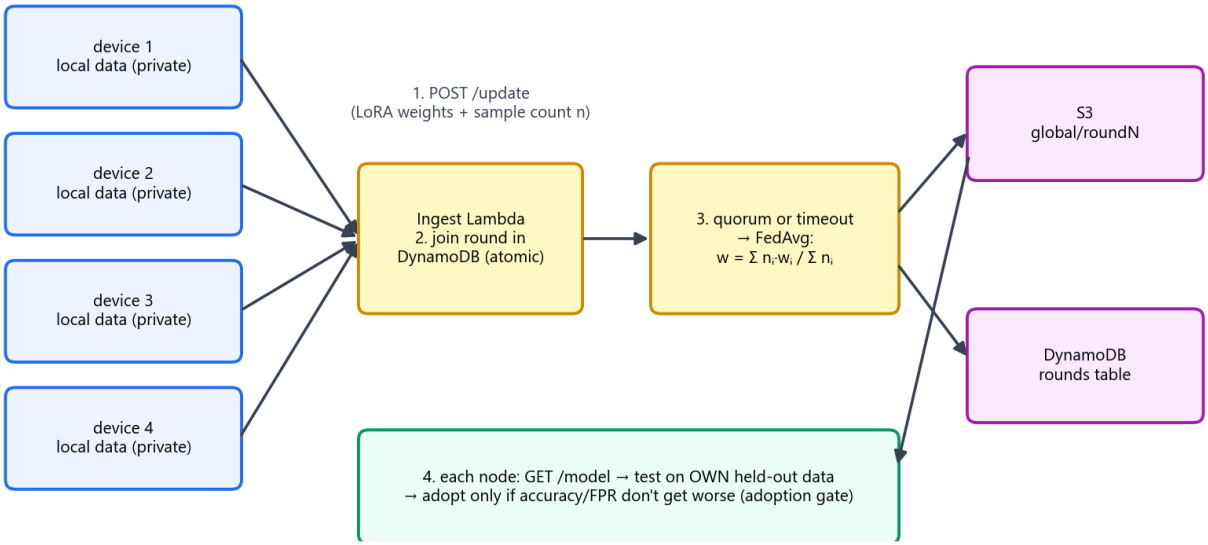


Figure 6. One federated round, end to end.

7.1 FedAvg (McMahan et al. 2017)

$$w_{global} = \Sigma_i\ n_i \cdot w_i\ /\ \Sigma_i\ n_i \quad (n_i = \text{number of local samples on device } i)$$

Applied to the LoRA adapter tensors only. As a robustness option we also implemented **coordinate-wise median** aggregation (take the median of each weight across devices), which resists a single poisoned client far better (Section 9).

7.2 Getting concurrency right

Lambda is stateless and 4 devices can upload at the same instant, so round state lives in DynamoDB and is changed with **conditional updates**: a device is added to *submitted* only if the round number still matches, and only one caller can flip the status from *collecting* to *aggregating* (optimistic locking). Exactly one aggregation happens per round, even if the timeout fires at the same moment quorum completes.

8. Edge nodes and the live demo

8.1 What runs on each node

- **4 × EC2 t3.small** (Amazon Linux 2023), managed only through SSM Session Manager (no SSH keys). Agent runs as the systemd service *fedgrid-agent*.
- **Live capture** with scapy on the public interface. A BPF filter keeps only *inbound bare SYN packets from outside the VPC*, i.e. connection attempts to the box. This means the agent can never block AWS's own traffic and lock itself out.
- **Real firewall**: blocks are actual *iptables -A INPUT -s <ip> -j DROP* rules (the service has CAP_NET_ADMIN / CAP_NET_RAW only, not full root).
- **Timing**: sync with cloud every 5-15 s; train + upload every 120 s aligned to the clock; round timeout 180 s; incident reports rate-limited per IP.
- **Policies differ per node on purpose**: devices 1-2 threshold, device 3 bandit, device 4 cost-aware, so the demo shows all three live.

8.2 The dashboard

React 18 + Vite + Recharts, deployed on **Cloudflare Pages** at **fedgrid.mohithkumar.dev**. It is **read-only**: a Cloudflare Pages Function proxies GET requests to API Gateway and adds the API key server-side, so the key is never in the browser. It shows an animated topology of the 4 nodes around the FedAvg core, KPI tiles, the live event stream (FL_UPLOAD, SYNC_DOWNLOAD, REJECT_GLOBAL, BLOCK) and the current round. A node turns red when it blocks, amber when it pre-blocks from peer intel, and grey after 5 minutes of silence.

8.3 Demo script (what the jury sees)

Act	What happens	What it proves
1. Federated, live	Round counter advances every ~2 min; 4 FL_UPLOAD rows per round; occasional REJECT_GLOBAL	Real federated learning running now; agents refuse bad models
2. Attack it	A phone or laptop port-scans a node (attack.ps1 / scanner app) → BLOCK with confidence + attention → iptables DROP → attacker times out	Autonomous detect-decide-act on real internet traffic
3. Fleet protection	Within ~9 s the other 3 nodes turn amber and have DROP rules for the same IP	Peer threat intel: one attacked, all protected
4. What it detects	Per-attack-class table from held-out test flows (Figure 8)	Accuracy comes from benchmarks, not the live demo

Bonus: real background internet scanners get blocked during the demo too; point them out when they appear.

9. Results (all measured on the 4 EC2 nodes, multiple seeds)

Metric	Result
Backbone accuracy, UNSW-NB15 official test set	87.7% (gradient-boosted trees: 90.1%)
Backbone accuracy, ToN_IoT	99.3%
Inference latency on the edge node	1.7 ms per flow
Upload size per round	10.3 KB with LoRA r=4 vs 103.7 KB full model
Model-replacement poisoning attack	FedAvg collapses to 47.5% accuracy; median aggregation holds 86.5%; adoption gate rejects every poisoned model
Benign traffic dropped by blocking (ToN_IoT)	49.8% (threshold, permanent blocks) → 12.5% (full agent)
Benign traffic dropped by blocking (UNSW-NB15)	22.1% → 2.1%, while letting fewer attacks through
Peer-intel propagation	median ≈ 9 s, range 1-12 s

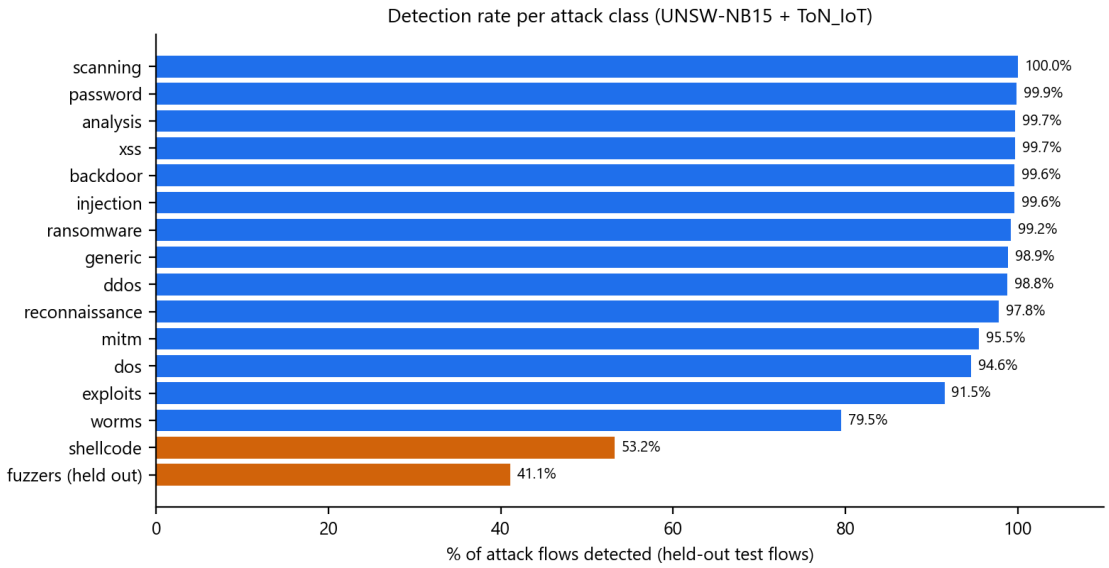


Figure 7. Detection rate per attack class on held-out flows. Fuzzers (held out of pretraining) and Shellcode are the weak spots.

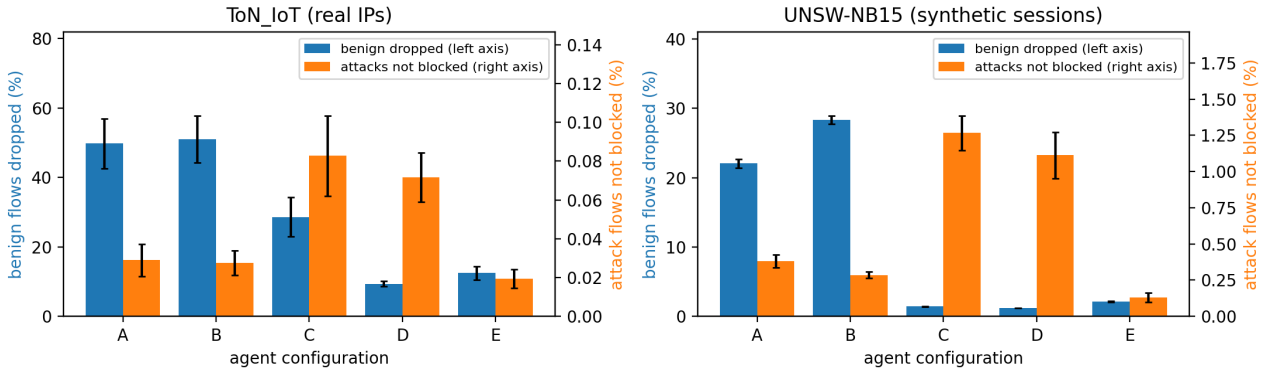


Figure 8. Agent configurations A (threshold, permanent block) → E (full agent). Benign drops fall sharply.

9.1 The honest negative result (know this before anyone asks)

Four federated rounds raise Fuzzers recall from 41.1% to 54.5%, which looks like federation taught the other nodes the new attack. **It didn't.** The false-positive rate also rose from 14.3% to 19.2%. If you simply move the frozen model's decision threshold to the same FPR you get 53.4%, and the ranking quality (Fuzzers-vs-benign AUROC) doesn't improve (0.823 vs 0.818). The "gain" is just a shift in operating point. More clients or 5× more local training don't change this. The full-model variant even hurts ranking (AUROC 0.962 → 0.950), while LoRA preserves it (0.959). We report this as a finding: many federated-IDS papers likely make this same mistake.

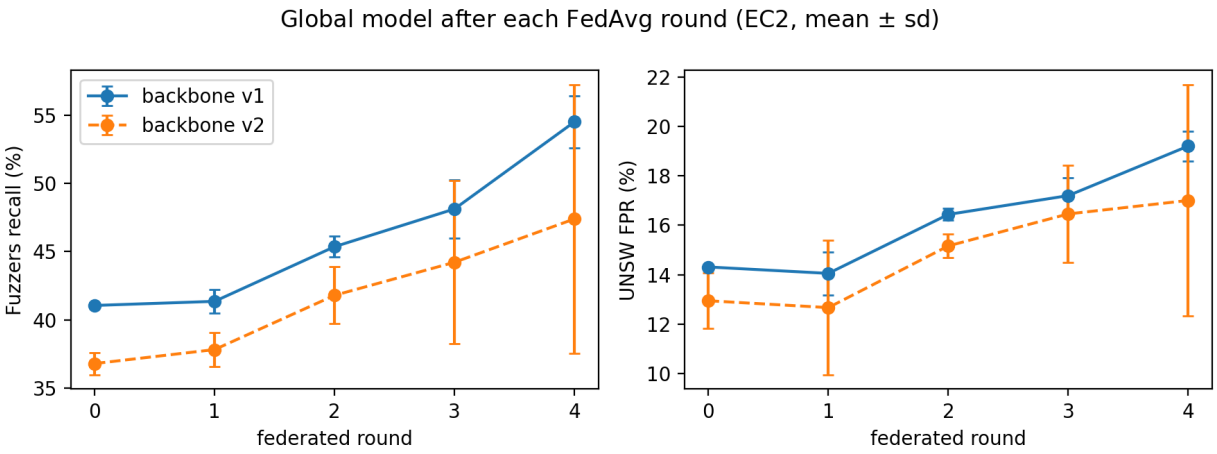


Figure 9. Fuzzers recall and FPR rise together over rounds: an operating-point shift, not new knowledge.

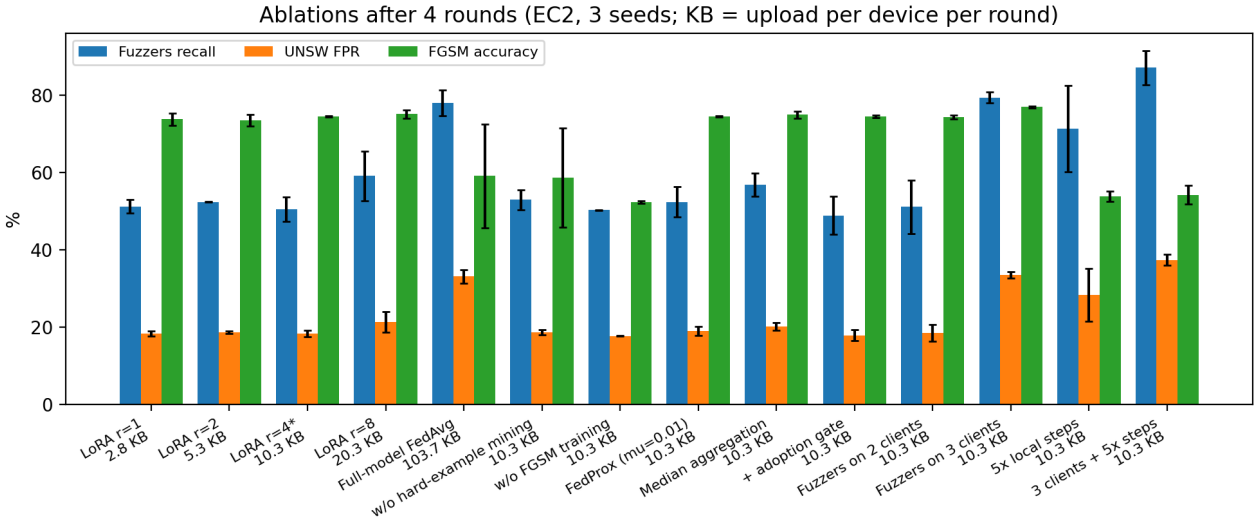


Figure 10. Ablations after 4 rounds (LoRA rank, full model, no mining, no FGSM, FedProx, median, gate, more clients/steps).

9.2 Bandit and investigator (simulation, 5 host splits)

With a simulated analyst that is right 95% of the time, the online bandit reaches 99.5% recall with a **block false-positive rate of 0.4%** (threshold rule: 1.4%) and the lowest cost per flow (0.067 vs 0.151 for the threshold rule), using about 8 LLM calls per 1,000 flows.

10. Every algorithm in one table

Algorithm	Used for	Key settings
Tabular Transformer	Attack detection: P(attack)	4 tokens, d=32, 2 layers, 4 heads, FFN 128
LoRA (Hu et al. 2021)	Tiny trainable adapter on frozen backbone	r=4, α=8, on FFN + head; via peft
FedAvg (McMahan et al. 2017)	Combine node adapters	Sample-size weighted mean, synchronous rounds
Coordinate-wise median	Poisoning-robust aggregation	Median per weight across devices
Dirichlet partition (Hsu et al. 2019)	Realistic non-IID data split	α = 0.5
Adam + OneCycle	Optimiser for pretraining / local training	lr 1e-3; batch 256 central, 64 local
Hard-example mining	Focus local training on mistakes	Loss-weighted sampling, cap 3×, floor 0.3
FGSM adversarial training	Robustness to evasion	ε = 0.05, clean + adversarial loss
Attention rollout (Abnar & Zuidema 2020)	Explain each detection	Product of residual-adjusted attention
Bayes cost-aware decision	Choose action by expected cost	Cost table above; analyst-feedback update
LinUCB (Li et al. 2010)	Learn the action policy online	α = 1, λ = 1, reward = −cost/20, exact federation
Exponential back-off blocking	Graded, self-expiring blocks	60 s × 2^(k−1), permanent after 4
Adoption gate	Refuse harmful global models	Δacc ≥ −1 pt and ΔFPR ≤ +2 pt
Optimistic locking (DynamoDB)	Exactly-once aggregation	Conditional updates on round + status
Gradient-boosted trees	Reference baseline	Same 8 features

11. Tech stack

Area	Technology
Language	Python 3.9 (edge nodes) / 3.x locally; JavaScript (dashboard)
ML	PyTorch 2.x, HuggingFace peft (LoRA), scikit-learn, NumPy, pandas
Packet capture & firewall	scapy (BPF filters), iptables (nft backend) on Linux
Edge compute	4 × AWS EC2 t3.small, Amazon Linux 2023, systemd service, SSM Session Manager
API	AWS API Gateway (REST) with per-device API keys and a usage plan
Compute (cloud)	AWS Lambda (Python) × 6 + EventBridge schedule
Storage	Amazon DynamoDB (round_state, rounds, incidents), Amazon S3 (adapters)
Infrastructure as code	AWS SAM (cloud/template.yaml)
AWS SDK	boto3; requests on the edge for HTTPS
Dashboard	React 18, Vite 5, Recharts, animated SVG topology
Dashboard hosting	Cloudflare Pages + Pages Functions (server-side API-key proxy), custom domain
Testing	moto (mocked DynamoDB/S3 for local simulation of the cloud)
Datasets	UNSW-NB15, ToN_IoT
Paper	LaTeX (IEEE format), matplotlib figures generated from EC2 logs

12. Limitations (say these before the jury does)

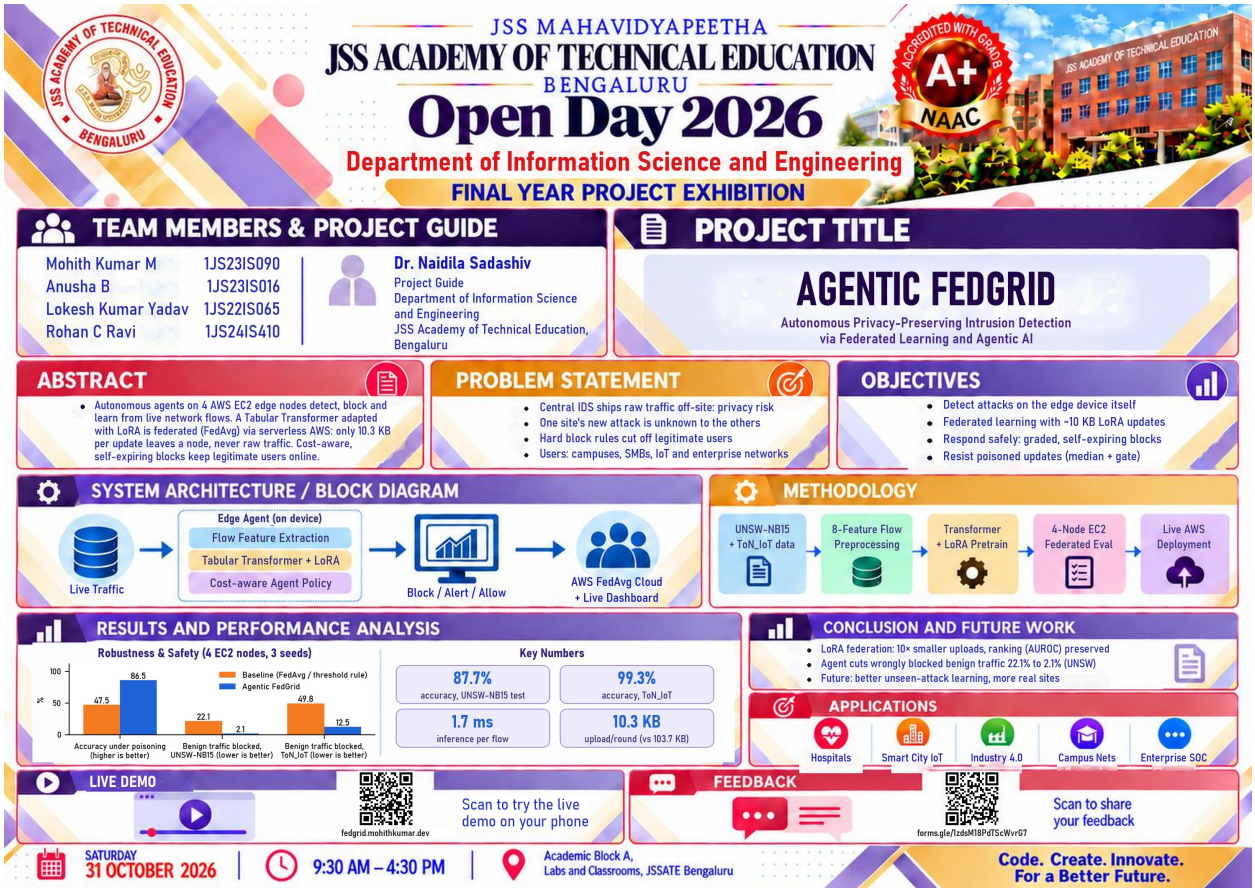
- **Live features ≠ training features.** The live sensor sees SYN packets; the model was trained on complete flow records. Live, it flags port scans well but also over-flags benign traffic from the same source. The live demo proves the **autonomous response pipeline**; the accuracy claims come from the benchmark test sets.
- **Federation does not teach the unseen attack** (Section 9.1).
- **Weak classes:** Fuzzers 41.1%, Shellcode 53.2%, Worms 79.5%.
- **Binary detector.** The attack name in demos comes from dataset labels or the weaker type classifier, not the detector.
- **LLM investigator** is built and simulated but not live.
- **Node-to-node traffic inside the AWS VPC is blocked by the platform**, so live attacks come from outside (phone or laptop) or from an on-box network namespace sandbox.
- **Demo-grade security.** Shared API keys and an open security group exist for the demo; keys must be rotated and the open rules removed afterwards.

13. Glossary

Term	Plain meaning
IDS / NIDS	(Network) intrusion detection system: software that spots attacks in traffic
Flow	One connection summarised as numbers (duration, bytes, packets, protocol…)
Federated learning	Many devices train one model together without sending their data anywhere
FedAvg	The cloud averages the devices' weights, weighted by how much data each has
LoRA adapter	A tiny add-on to a frozen model; the only part trained and shared
Non-IID	Each device's data looks different (different attack mix), like the real world
FPR	False-positive rate: share of normal traffic wrongly flagged as an attack
Recall	Share of real attacks that were caught
AUROC	How well the model ranks attacks above normal traffic, independent of threshold
Poisoning	A malicious device uploads bad weights to wreck the shared model
Contextual bandit	A learner that picks an action from context and improves from rewards
Attention rollout	A way to read which input features a Transformer relied on

14. Where things live in the repo

Path	Contents
model/	tabular_transformer.py (model + LoRA), train_backbone.py, evaluate*.py, explain.py (rollout), type classifier, GBM reference
edge_agent/	agent.py (EdgeAgent loop), policy.py (cost rule, gate), bandit.py (LinUCB), actuator.py (graded iptables), trainer.py (local LoRA + FGSM), comms.py (API client), live_capture.py (scapy), flow_history.py, investigator.py, drift_monitor.py
cloud/	template.yaml (SAM), 6 Lambda handlers, shared/aggregate.py (FedAvg/median), store.py (DynamoDB/S3), serialization.py (wire format)
dashboard/	React app (Topology, KpiStrip, LiveStream, IncidentFeed…) + functions/ (Cloudflare proxy)
data/scripts/partition_data.py	Canonical features, vocab, Dirichlet split, Fuzzers hold-out
paper_figures/, paper_latex/	Paper, tables and figures generated from EC2 logs
run_edge_node.py, DEMO_RUNBOOK.md	Node entry point and the step-by-step demo script



Appendix. Our Open Day poster.